

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
<NAME OF INSTITUTE / UNIVERSITY, CITY>

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

*Project Title: SmartScreener: AI-Powered Resume Screening and Candidate
Ranking System*

Standard Academic Compliance: IEEE Std 830-1998

SUBMITTED BY:

1. Arun Avasthi (Roll No: 2400321540051)

Degree: Bachelor of Technology (B.Tech)

Academic Year: 2026 - 2027

UNDER THE GUIDANCE OF:

Prof. / Dr. <Ankit Mishra>

<Data scientist>

Department of Computer Science & Engg(ds)

Status: 3rd Year Project Specification

Date of Submission: <01-10-2026>

Document Revision & Approval History

0.1	15-Sep-2026	Initial Draft & Scope Definition	Smartscreener	Ankit mishra
1.0	30-Sep-2026	Final 3rd Year Mid-Term SRS Baseline	Smartscreener	HCL TECH

Guidance Note: This Software Requirements Specification document conforms to the IEEE 830-1998 standard recommended practice. All sections are strictly designed for 3rd-year university lab evaluation, project reviews, and system design baselines.

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to provide a complete, clear, and unambiguous definition of the functionality, external interfaces, performance targets, and constraints of the proposed system: **'SmartScreener: AI-Powered Resume Screening and Candidate Ranking System'**. This specification serves as a binding reference agreement between student developers, project evaluators, academic supervisors, and prospective end users during the 3rd-year engineering curriculum.

1.2 Scope of the System

The proposed application is designed to solve *the time-consuming, inconsistent and keyword-dependent manual screening of large volumes of resumes against a job description* by delivering an end-to-end automated platform. Key boundary guidelines include:

- **In-Scope Capabilities:** Centralized data ingestion, user authentication, role-based workflows, automated inference/business calculation, and analytical dashboard visualization.
- **Out-of-Scope Boundaries:** Hardware fabrication beyond basic microcontroller interfacing, external enterprise ERP integrations, and live commercial payment processing (mock sandbox mode utilized).
- **Expected Benefits:** Reduction of operational turnaround time by 60%, improved audit accuracy, elimination of redundant manual logging, and immediate accessibility across devices.

1.3 Definitions, Acronyms, and Abbreviations

Standard	SRS	Software Requirements Specification (IEEE Std 830-1998 format).
Architecture	API	Application Programming Interface for data interchange via JSON/HTTPS.
Project Specific	JD	Job Description - the textual specification (title, responsibilities, required skills, years of experience) against which resumes are screened.
Project Specific	ATS	Applicant Tracking System - recruitment software that collects and filters job applications; SmartScreener augments this step with semantic scoring.
Project Specific	LLM / Ollama	Large Language Model served locally through Ollama (REST API on port 11434, model "mistral") for structured resume information extraction.
Project Specific	NLP Fallback	Rule-based extractor (regular expressions and a built-in skills dictionary) used automatically when the Ollama engine is unavailable.
Project Specific	Cosine Similarity	Vector-space measure (0 to 1) of textual similarity between resume text and job description using term-frequency vectors.
Architecture	FastAPI / Next.js	Python ASGI framework for the backend REST API / React-based framework for the web dashboard (with Tailwind CSS and Recharts).
Database	SQLite	Embedded, file-based relational database (db.sqlite) used for persistent storage of job pipelines and candidate records.

1.4 References

- **IEEE Std 830-1998:** IEEE Recommended Practice for Software Requirements Specifications.
- **Software Engineering (10th Edition):** Ian Sommerville, Pearson Education.

- **Relevant Research Papers & APIs:** SmartScreener source repository (<https://github.com/arun-avasthi/smartscreener>); C. D. Manning, P. Raghavan and H. Schütze, Introduction to Information Retrieval, Cambridge University Press, 2008 (vector space model and cosine similarity); FastAPI documentation (<https://fastapi.tiangolo.com>); Next.js documentation (<https://nextjs.org/docs>); Ollama API documentation (<https://github.com/ollama/ollama/blob/main/docs/api.md>); SQLite documentation (<https://www.sqlite.org/docs.html>); pypdf and python-docx documentation; Recharts documentation (<https://recharts.org>).

1.5 Document Overview

The remainder of this SRS is partitioned into structural sections: Section 2 outlines the product perspective, target user personas, operational context, and design constraints. Section 3 itemizes detailed functional requirements organized into logical software modules. Section 4 specifies external interfaces (UI, Hardware, Software, Network). Section 5 defines non-functional attributes (Performance, Reliability, Security). Finally, Section 6 provides standard academic modeling diagrams.

2. Overall Description

2.1 Product Perspective

The project is structured as a self-contained multi-tier system (comprising a presentation layer, an application/API logic tier, and a persistent database backend). It interfaces with external services (such as cloud storage or messaging gateways) strictly through secure, authenticated RESTful endpoints.

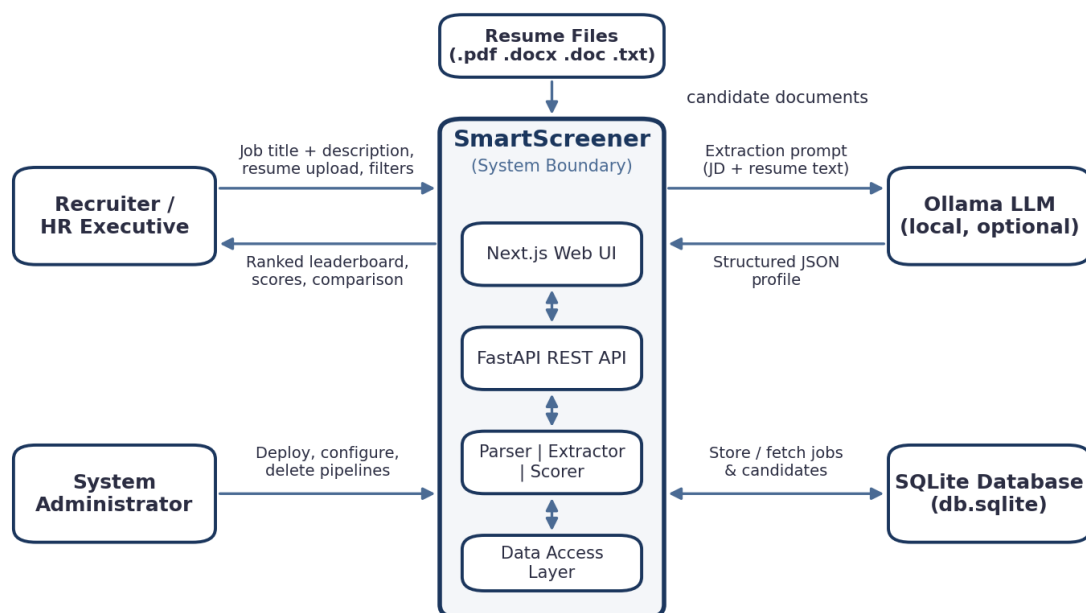


Figure 2.1 - System Context Diagram (DFD Level 0) of SmartScreener

2.2 User Classes and Characteristics

The target software accommodates distinct user personas with varying access hierarchies:

System Administrator	High (DevOps / DB Management)	Complete access: deployment and configuration of the FastAPI/Next.js services and the Ollama LLM engine, master database (db.sqlite) backup/restore, removal of screening pipelines; user role assignment and audit logs are planned for a future release.
Registered End User	Intermediate (General Web User)	Recruiter / HR Executive: creates job specifications and screening pipelines, uploads resumes in batches, reviews the ranked leaderboard and AI verdict summaries, compares up to three candidates, and deletes pipelines.
Guest / Public Visitor	Basic	Read-only access to the public landing view and system overview; no access to job pipelines, candidate data or resumes. A self-registration portal is planned for a future release.

2.3 Operating Environment

The application operates within the following software and hardware environments:

- **Client Requirements:** Modern web browser (Google Chrome $\geq$ v110, Mozilla Firefox $\geq$ v108, Edge, Safari); minimum display resolution 1366x768 for desktop, responsive view down to 360x640.
- **Server Runtime Environment:** Linux (Ubuntu 22.04 LTS / Debian 12) or Windows Server; Python 3.10+ (FastAPI/Uvicorn backend) and Node.js 20 LTS or later (Next.js frontend); optional local Ollama service on port 11434.
- **Database Storage Tier:** SQLite 3 (embedded file db.sqlite, no separate database server process); scalable to PostgreSQL 15 / MySQL 8.0 in future releases for multi-user deployments.

2.4 Design and Implementation Constraints

- **Academic Budget Constraint:** Utilization of free-tier cloud resources (e.g., Render, Railway, AWS Educate, or local containerized hosting).
- **Security Compliance:** Passwords stored exclusively as salted hashes (PBKDF2 or Bcrypt); token invalidation upon user logout.
- **Modularity Standards:** Adherence to Model-View-Controller (MVC) or modular REST service patterns to ensure clean separation of concerns.

2.5 Assumptions and Dependencies

- **Assumptions:** Users possess continuous Internet connectivity with minimal bandwidth of 2 Mbps; client clocks are synchronized via NTP.
- **Dependencies:** Continuous availability of third-party public APIs (e.g., the locally hosted Ollama REST API at <http://localhost:11434> serving the "mistral" model; the system degrades gracefully

to Local NLP Fallback mode when it is unavailable) and of open-source libraries (FastAPI, pypdf, python-docx, Next.js, Recharts).

3. System Features and Functional Requirements

Requirements are categorized by functional subsystem modules with unique requirement identifiers (FR-x.x):

3.1 Module 1: Authentication and Access Governance

FR-1.1	User Account Registration	Email, Username, Password (≥ 8 chars, alphanumeric + symbol)	High
FR-1.2	Secure Credential Login	Registered Email/User + Plain Password; verifies bcrypt hash	High
FR-1.3	Session Token Management	Generates signed JWT (HS256) valid for 120 minutes	Medium

3.2 Module 2: Core Processing & Business Operations

FR-2.1	Primary Data Ingestion / Input Capture	Job title + description form (both mandatory); batch resume upload (.PDF / .DOCX / .DOC / .TXT) via file picker or drag-and-drop	High
FR-2.2	Automated Processing / Inference Engine	Local LLM extraction (Ollama, JSON mode, temperature 0.1, 8 s timeout) with automatic rule-based NLP fallback; weighted score calculation	High
FR-2.3	Persistent Data Records Storage	Persist jobs and candidates in SQLite; candidates.job_id references job_descriptions.id with cascade delete	High
FR-2.4	Resume Text Parsing	PDF via pypdf; DOCX/DOC via python-docx (paragraphs and tables); other files read as UTF-8 text; whitespace normalized; files with < 50 characters of text skipped	High
FR-2.5	Job Requirement Extraction	Required skills matched from JD text against the built-in skills dictionary; fallback to up to 4 title keywords (> 3 characters)	High
FR-2.6	Candidate Profile Extraction	Name, email, phone, skills, total years of experience, roles, companies, highest education and 2-sentence summary	High
FR-2.7	Weighted Match Scoring	Technical 40% (70% skill coverage + 30% cosine similarity); Experience 40% (candidate years / JD target years, capped at 100); Education 20% (PhD 100, Master 90, Bachelor 80, Associate 70, else 60); overall rounded to 1 decimal	High
FR-2.8	Batch Fault Tolerance	An unreadable or failing resume is logged and skipped without aborting the remaining files in the batch	Medium

3.3 Module 3: Reporting, Visualization & Administrative Control

FR-3.1	Analytical Metric Dashboard	Ranked leaderboard (score bands: >=80 high, 50-79 medium, <50 low), per-candidate score bar chart, pipeline history list, engine status indicator	Medium
FR-3.2	Exportable Summary Reports	Generates structured PDF or XLSX downloads of records	Low
FR-3.3	User Status Moderation (Admin)	Toggle active/disabled user status; manage platform parameters	High
FR-3.4	Candidate Search, Filter and Sort	Search by name/email; filter by skill; sort by overall score or years of experience (ascending/descending)	Medium
FR-3.5	Candidate Detail View	Contact details, skills, education, experience timeline, AI verdict summary, score chart and raw extracted resume text	Medium
FR-3.6	Candidate Comparison	Select 2 to 3 candidates of one pipeline for a side-by-side matrix of scores, skills and summaries	Medium
FR-3.7	Pipeline Management	Create, list, select and delete job pipelines; deletion (with confirmation) removes associated candidates	Medium
FR-3.8	System Health Check	GET /api/health reports API status and LLM engine mode (connected / fallback_mode)	Low

4. External Interface Requirements

4.1 User Interfaces (UI)

The interface will follow standard UX usability best practices (clean navigation bar, breadcrumbs, modal validation dialogs). All forms must provide immediate visual feedback (success banners or inline error alerts under erroneous fields).

SmartScreener | AI Talent Screening Platform Engine Status Active Pipeline v

Job Specification

Upload Resumes

Drag & drop resumes (.pdf .docx .doc .txt)

Existing Pipelines (3)

Ranking Leaderboard (n)

[] #1	Candidate A	Skills: chips...	Exp: x yrs	91.5
[] #2	Candidate B	Skills: chips...	Exp: x yrs	84.0
[] #3	Candidate C	Skills: chips...	Exp: x yrs	72.3
[] #4	Candidate D	Skills: chips...	Exp: x yrs	58.9
[] #5	Candidate E	Skills: chips...	Exp: x yrs	41.2

(1) Screening Workspace - Job Specification, Resume Upload, Pipelines and Ranking Leaderboard

4.2 Hardware Interfaces

Specification of physical interaction points:

- **Standard Deployment:** Standard input/output peripherals (Keyboard, Mouse, Display $\geq$ 720p).
- **Embedded / IoT Variants (if applicable):** Not applicable; SmartScreener is a purely software system with no microcontroller, sensor or serial-port interfaces.

4.3 Software Interfaces

- **Relational Database Connector:** Python standard-library sqlite3 module (parameterized queries, no ORM) interfacing with the local SQLite database file; Pydantic models validate API requests and responses.
- **External Cloud Utilities:** Ollama local REST API (GET / availability probe and POST /api/chat for structured extraction) over HTTP on port 11434; pypdf and python-docx libraries for resume text extraction. No cloud storage services are used.

4.4 Communication Interfaces

- **Web Traffic Protocol:** HTTPS (Hypertext Transfer Protocol Secure over Port 443 with TLS 1.3).
- **Data Serialization:** All payload bodies exchanged using standard UTF-8 encoded application/json format; resume file uploads use multipart/form-data.

5. Non-Functional Requirements (NFRs)

5.1 Performance	Page load latency $<$ 2.0s; API response time $<$ 350ms under concurrency of 50 users.	JMeter / Apache Benchmark load testing.
5.2 Security	Zero plain-text passwords; parameterized queries to neutralize SQL injection; rate limiting (100 req/min).	OWASP ZAP dynamic vulnerability scan; manual penetration test.
5.3 Reliability	System uptime $\geq$ 98% during academic review; automatic session recovery on client disconnect.	Stress testing & server process monitor (PM2 / systemd auto-restart).
5.4 Portability	Cross-browser compatibility (Chrome, Edge, Firefox, Safari); responsive viewport (mobile to desktop).	Responsive viewport inspection in Developer Tools and BrowserStack.

6. System Design and Analysis Models

University evaluation rubrics require standard graphical analysis models illustrating the dynamic and static behavior of the application:

Use Case Diagram	Maps user actors (Admin, User, Visitor) to discrete functional interactions.	UML Standard diagram + Use Case specification tables.
Data Flow Diagram (DFD)	Illustrates information flow: Level 0 (Context) and Level 1 (Modular decomposition).	Gane-Sarson / Yourdon & Coad notation flow chart.
Entity-Relationship (ERD)	Details entities, primary/foreign keys, attributes, and relationships.	Crow's Foot notation schema with 3NF normalization notes.
Activity / Sequence Diagram	Chronological sequence of messages between UI, Controller, and DB for core feature.	UML Sequence diagram detailing happy-path and error recovery.

7. Academic Review & Sign-Off

This Software Requirements Specification document has been submitted and reviewed by the academic supervisory panel:

Signature: ARUN AVASTHI
Date: 01-10-2026

Signature: _____
Date: _____

Signature: _____
Date: _____